

Extending Multilingual Language Models to Low-Resource Languages

A Tutorial on Glot500-m, Vocabulary Extension, and Continued Pretraining

Case Study: Coptic and Syriac

Tutorial Notes

May 22, 2026

Abstract

This tutorial walks through the methodology of extending a multilingual language model to new low-resource languages, using Glot500-m as the base model and Coptic and Syriac as case studies. We begin with the foundations of multilingual pretraining, move through tokenization and vocabulary extension, discuss embedding initialization strategies, examine training methodologies (full fine-tuning vs. LoRA), and conclude with evaluation protocols. Throughout, we maintain mathematical precision about probability distributions and acknowledge open questions about how cross-lingual alignment actually works in shared embedding spaces.

Contents

1	Background: The Multilingual Pretraining Landscape	3
1.1	The Scope Problem	3
1.2	What Glot500-m Is	3
1.3	The Gap: Coptic and Syriac	3
2	Estimating Available Data for Coptic and Syriac	3
2.1	Coptic	4
2.2	Syriac	4
3	The Sampling Strategy: Temperature-Scaled Multinomial	4
3.1	The Sampling Problem	4
3.2	Two-Stage Hierarchical Sampling	4
3.3	The Multinomial Distribution	5
3.4	Intuition for α	5
3.5	Conditional probability (Stage 2)	5
3.6	Joint probability (chain rule)	5
3.7	The Two-Faced Effect of $\alpha < 1$	5
4	Tokenization and Vocabulary Extension	6
4.1	Why the New Tokenizer Matters	6
4.2	The Glot500 Vocabulary Extension Procedure	6
4.3	Three Vocabularies in Glot500: Clarification	6
4.4	Why SentencePiece Unigram for Word Counting	6
5	Embedding Initialization Strategies	7
5.1	Random Initialization	7
5.2	Mean Initialization	7
5.3	FOCUS Initialization	7
5.4	Align Initialization	7

5.4.1	The Core Idea	7
5.4.2	Setup and Notation	8
5.4.3	Step 1 — Dual Tokenization	8
5.4.4	Step 2 — Per-Occurrence Correspondence	8
5.4.5	Step 3 — Aggregation Across the Corpus	8
5.4.6	Step 4 — The Embedding Formula	9
5.4.7	Worked Example	9
5.4.8	Why Align Outperforms Mean	9
5.4.9	Edge Cases	10
5.4.10	Why Align Wins in Low-Resource Settings	10
5.4.11	Algorithm Summary	11
5.4.12	Practical Recommendation for Coptic/Syriac	11
5.5	When Initialization Strategy Matters	11
6	Continued Pretraining: Full Fine-Tuning vs. LoRA	12
6.1	Glott500’s Approach: Full Fine-Tuning	12
6.2	Why Glott500 Chose Full Fine-Tuning	12
6.3	Yamaguchi’s Approach: LoRA + Embedding Full Update	12
6.4	Decision Matrix for the Coptic/Syriac Project	12
6.5	Training Data Mixture	13
7	SimAlign: Word-Level Cross-Lingual Alignment	13
7.1	The Idea	13
7.2	The Algorithm	13
7.3	Why the Intersection Condition	14
7.4	Why SimAlign Works	14
8	How Does Cross-Lingual Alignment Actually Work?	14
8.1	The Distributional Hypothesis Reformulated	14
8.2	The Bridging Mechanisms	14
8.2.1	Real anchor tokens	14
8.2.2	Subword overlap	14
8.2.3	Incidental bilingual content	15
8.2.4	Shared parameters	15
8.2.5	Structural isomorphism	15
8.3	Implications for Coptic and Syriac	15
8.4	Open Research Questions	15
9	Evaluation: Glott500’s Four Tasks	15
9.1	Roundtrip Alignment	15
9.2	Sequence Labeling	16
9.3	Sentence Retrieval	16
9.4	Text Classification	16
9.5	What “Zero-Shot” Means Here	16
10	Putting It Together: The Coptic/Syriac Project	16
10.1	Workflow	16
10.2	Experimental Design	17
10.3	Project Framing	17
11	Summary and Key Takeaways	17

1 Background: The Multilingual Pretraining Landscape

1.1 The Scope Problem

The NLP community has historically focused on roughly 100 languages, with most attention on high-resource languages like English, Chinese, and the major European languages. Models like multilingual BERT (mBERT) and XLM-R cover about 100 languages, but thousands of the world’s languages remain unsupported. Among the most underserved are ancient and liturgical languages with substantial written traditions: Coptic, Classical Syriac, Ge’ez¹, and many others.

Glott500-m [1] addresses this gap by scaling *horizontally* rather than vertically: instead of building larger models for the same 100 languages, it extends XLM-R-base to cover 511 languages, most of them low-resource. This is achieved through:

1. Collecting Glott500-c, a corpus covering 511 languages.
2. Extending XLM-R’s vocabulary with new tokens.
3. Continued pretraining with the masked language modeling (MLM) objective.

1.2 What Glott500-m Is

- **Base architecture:** XLM-R-base (270M parameters, 12 layers, 768 hidden dim).
- **Vocabulary:** 401K tokens (XLM-R’s 250K + 151K new tokens).
- **Training data:** Glott500-c, hundreds of GB across 511 language-scripts.
- **Inclusion threshold:** A language-script is included only if it has more than 30,000 sentences after cleaning.
- **Repository:** <https://github.com/cisnlp/Glott500>
- **Paper:** <https://arxiv.org/abs/2305.12182>

1.3 The Gap: Coptic and Syriac

Important: Glott500’s coverage limitation

Despite covering 511 languages, Glott500-c does *not* include Coptic (cop) or Classical Syriac (syc). This is not because the languages lack sufficient text—both have substantial written traditions—but because Glott500’s automatic collection pipeline relied primarily on web crawls (CC100, OSCAR, mC4) and Wikipedia, which poorly cover ancient and liturgical languages.

This gap motivates a substantive research project: collecting Coptic and Syriac corpora from *academic* sources and extending Glott500-m to cover these languages. The project becomes a methodological critique of automatic data collection pipelines as well as a practical model extension.

2 Estimating Available Data for Coptic and Syriac

A practical question: can we even reach the 30K sentence threshold (and ideally exceed it) for these languages?

¹Ge’ez is an ancient Ethiopian Semitic language, extinct as a spoken language for about a millennium but still the liturgical language of Ethiopian and Eritrean Orthodox Christianity. It has substantial literature (including unique preservation of ancient texts like the complete Book of Enoch)

2.1 Coptic

- **Coptic Scriptorium** (<https://copticcriptorium.org/>): Academic project with annotated Sahidic Coptic texts. TEI XML format.
- **Coptic Bible**: Sahidic and Bohairic versions of New Testament ($\approx 8K$ verses) plus surviving Old Testament fragments.
- **Patristic literature**: Shenoute corpus (large 4th-5th century author), Pachomian literature, Apophthegmata Patrum (Sayings of the Desert Fathers), Nag Hammadi texts.

Estimated total: 25,000–40,000 sentences (approaches or meets the threshold).

2.2 Syriac

- **Peshitta** (Syriac Bible): Complete Old and New Testament, $\approx 30K$ verses—already meets the threshold by itself.
- **Patristic Syriac literature**: Ephrem the Syrian (4th century), Aphrahat’s Demonstrations, Isaac of Nineveh, Jacob of Edessa, Bar Hebraeus.
- **Translation literature**: Syriac translations of Greek patristic and philosophical works.
- **Resources**: Beth Mardutho, CAL (Comprehensive Aramaic Lexicon), SEDRA database.

Estimated total: 80,000–130,000 sentences (well exceeds the threshold).

Implication

Both languages have enough data to be included in a multilingual model. The Glot500 team’s omission likely stems from pipeline limitations (web-focused collection) rather than data scarcity. The project framing should emphasize this: *filling a gap left by automatic collection methods*.

3 The Sampling Strategy: Temperature-Scaled Multinomial

3.1 The Sampling Problem

When training a multilingual model, we need to pick sentences to train on. If we sample proportionally to corpus size, head languages (like English with billions of tokens) will dominate, and tail languages will see almost no training signal. We need a strategy that gives tail languages a boost.

3.2 Two-Stage Hierarchical Sampling

The standard approach (XLM-R, Glot500) is hierarchical:

1. **Stage 1 (Language sampling)**: Pick a language L_i according to some distribution $\pi_i = p(L = L_i)$.
2. **Stage 2 (Sentence sampling)**: Given the chosen language, pick sentences uniformly from its corpus.

3.3 The Multinomial Distribution

The Stage 1 distribution uses a temperature parameter $\alpha \in [0, 1]$:

$$\pi_i = p(L = L_i) = \frac{N_{L_i}^\alpha}{\sum_j N_{L_j}^\alpha} \quad (1)$$

where N_{L_i} is the number of sentences in language L_i 's corpus.

3.4 Intuition for α

- $\alpha = 1$: Raw proportional sampling. Head languages dominate.
- $\alpha = 0$:

$$\pi_i = p(L = L_i) = \frac{1}{\sum_j N_{L_j}} \quad (2)$$

Uniform over languages. All languages equally likely.

- $\alpha = 0.3$: Standard choice (XLM-R, Glot500). Balanced compromise.

Example 3.1 (Concrete effect of $\alpha = 0.3$). *Consider two languages:*

- *Language A: $N_A = 10,000,000$ sentences (English-like).*
- *Language B: $N_B = 10,000$ sentences (Coptic-like).*

Raw ratio: A has 1000× more data than B.

With $\alpha = 1$: $\pi_A : \pi_B = 10^7 : 10^4 = 1000 : 1$.

With $\alpha = 0.3$: $\pi_A \propto (10^7)^{0.3} \approx 631$, $\pi_B \propto (10^4)^{0.3} \approx 79$. So $\pi_A : \pi_B \approx 8 : 1$.

With $\alpha = 0$: $\pi_A : \pi_B = 1 : 1$.

So $\alpha = 0.3$ reduces the 1000× ratio to about 8×—tail languages get much more training signal while head languages still dominate slightly.

3.5 Conditional probability (Stage 2)

Given language L_i , each of its N_{L_i} sentences is equally likely:

$$p(s \mid L = L_i) = \begin{cases} \frac{1}{N_{L_i}} & \text{if } s \in \text{corpus}(L_i) \\ 0 & \text{otherwise} \end{cases} \quad (3)$$

3.6 Joint probability (chain rule)

For sentence s belonging to L_i :

$$p(s, L = L_i) = p(s \mid L = L_i) \cdot p(L = L_i) = \frac{1}{N_{L_i}} \cdot \frac{N_{L_i}^\alpha}{\sum_j N_{L_j}^\alpha} = \frac{N_{L_i}^{\alpha-1}}{\sum_j N_{L_j}^\alpha} \quad (4)$$

3.7 The Two-Faced Effect of $\alpha < 1$

There are two statements that sound contradictory but both hold:

1. **Language-level:** Larger corpora are sampled *more often*. $\pi_i \propto N_{L_i}^\alpha$ with $\alpha > 0$.
2. **Sentence-level:** Individual sentences from larger corpora are sampled *less often*. $p(s) \propto N_{L_i}^{\alpha-1}$ with $\alpha = 0.3$ gives exponent $\alpha - 1 = -0.7$.

Example 3.2 (Intuitive analogy). *A large city is visited more often than a small village (Statement 1), but a specific individual in the large city is encountered less often than a specific individual in the village, because the city's "visiting probability" is divided among many residents (Statement 2). Both statements coexist without contradiction.*

4 Tokenization and Vocabulary Extension

4.1 Why the New Tokenizer Matters

Adding a new language to a multilingual model typically requires extending the vocabulary. Without new tokens in the new vocabulary, the existing tokenizer will fragment the new language into many short subwords (often individual characters), producing inefficient and uninformative representations.

4.2 The Glot500 Vocabulary Extension Procedure

Glott500 uses SentencePiece with the unigram language model (matching XLM-R’s tokenizer algorithm). The pipeline:

1. Train an auxiliary SentencePiece tokenizer on the new corpus plus the old corpus.
2. Identify tokens in the auxiliary vocabulary that are not in the original tokenizer.
3. Merge the new tokens into the original vocabulary.
4. Resize the model’s embedding matrix to accommodate new tokens.
5. Initialize embedding vectors for the new tokens.

4.3 Three Vocabularies in Glot500: Clarification

Three different 250K vocabularies

1. **XLM-R’s tokenizer:** 250K vocab trained on CC-100 (100 languages). The fixed input to Glot500.
2. **Auxiliary tokenizer:** A new 250K vocab trained on Glot500-c (500+ languages) just to *discover* new tokens. Will be disposed after the new tokens are computed.
3. **Final Glot500-m tokenizer:** $401K = 250K$ (from XLM-R) + 151K genuinely new tokens. About 100K tokens from the auxiliary vocabulary overlapped with XLM-R’s vocab and were discarded.

4.4 Why SentencePiece Unigram for Word Counting

A subtle point: Glot500 uses SentencePiece for *word counting* in its data cleaning filters (SF1-SF5). This is because:

- Many of the 2266 languages lack reliable word tokenization conventions.
- Languages without whitespace (Chinese, Japanese, Thai, Burmese) cannot be word-counted via whitespace tokenization.
- SentencePiece (with the NLLB tokenizer, Costa-jussà et al. 2022) provides a uniform proxy for “word” across all languages.

So “word” is operationally defined as either a whitespace-delimited token (when possible) or a SentencePiece subword.

5 Embedding Initialization Strategies

When new tokens are added to the vocabulary, their embedding vectors [as parameters to be learned] must be initialized. The choice of initialization significantly affects how quickly the model learns useful representations for these tokens.

5.1 Random Initialization

Glott500 uses random initialization. This works because Glott500 has hundreds of GB of training data—the model has enough signal to learn good embeddings from scratch.

When random works: Large data, sufficient compute, willingness to train extensively.

5.2 Mean Initialization

Initialize each new token’s embedding as the mean of the embeddings of the subwords the original tokenizer would have produced for this string.

```
1 import torch
2
3 for new_token_id in new_token_ids:
4     new_token = tokenizer.convert_ids_to_tokens(new_token_id)
5     # How would the original tokenizer have split this string?
6     original_subwords = original_tokenizer.tokenize(new_token)
7     original_ids = original_tokenizer.convert_tokens_to_ids(
8         original_subwords)
9     original_embeddings = model.get_input_embeddings()(torch.tensor(
10         original_ids))
11     # Use the mean as the new embedding
12     new_embedding = original_embeddings.mean(dim=0)
13     model.get_input_embeddings().weight.data[new_token_id] =
14     new_embedding
```

Listing 1: Mean initialization for new tokens

5.3 FOCUS Initialization

FOCUS (Dobler & de Melo, 2023) initializes new token embeddings using a weighted combination of similar tokens from the original vocabulary, based on auxiliary fastText embeddings.

5.4 Align Initialization

Align (Yamaguchi et al., 2024) initializes each new token’s embedding using a **frequency-weighted sum of source embeddings**, where the weights come from observing how the source/original tokenizer naturally decomposes the new token’s character span across the target corpus.

5.4.1 The Core Idea

Unlike Mean—which decomposes each new token in *isolation* by re-tokenizing its surface string once—Align observes that the same new token’s character span can be tokenized differently by the source/original tokenizer depending on the surrounding context. By collecting these contextual decompositions across the entire target corpus and weighting them by frequency, Align captures information that Mean discards.

5.4.2 Setup and Notation

Inputs:

- T_s : source tokenizer with vocabulary V_s (e.g., Glot500-m’s original tokenizer).
- T_t : target (extended) tokenizer with vocabulary $V_t = V_s \cup V_{\text{new}}$.
- V_{new} : set of new tokens added by extension.
- $D = \{x_1, x_2, \dots, x_N\}$: target-language corpus (Coptic + Syriac in our case).
- $\mathbf{e}_{t'} \in \mathbb{R}^d$: the existing embedding for each source token $t' \in V_s$.

Output: An initial embedding $\mathbf{e}_t \in \mathbb{R}^d$ for every new token $t \in V_{\text{new}}$.

5.4.3 Step 1 — Dual Tokenization

For each sentence $x_n \in D$, compute both segmentations in parallel:

$$T_t(x_n) = [\tau_1, \tau_2, \dots, \tau_{L_n}], \quad \tau_j \in V_t \quad (5)$$

$$T_s(x_n) = [\sigma_1, \sigma_2, \dots, \sigma_{M_n}], \quad \sigma_k \in V_s \quad (6)$$

Both segmentations cover the same characters but chunk them differently.

5.4.4 Step 2 — Per-Occurrence Correspondence

For each position j in $T_t(x_n)$ where $\tau_j \in V_{\text{new}}$, identify the character span $[a_j, b_j]$ that τ_j covers in x_n . Extract the contiguous slice of $T_s(x_n)$ covering exactly the same span:

$$\mathbf{t}_{n,j} = (\sigma_k, \sigma_{k+1}, \dots, \sigma_{k+m}) \quad (7)$$

where $c_k = a_j$ and $d_{k+m} = b_j$ (c_k, d_k are the character endpoints of σ_k).

This $\mathbf{t}_{n,j}$ is the **corresponding source tuple** for this occurrence of τ_j . It is the source-vocabulary “spelling” of the new token’s span, observed in real corpus context.

5.4.5 Step 3 — Aggregation Across the Corpus

For each new token $t \in V_{\text{new}}$, collect all corresponding source tuples observed across the entire corpus:

$$\mathcal{C}_t = \{\mathbf{t}_{n,j} \mid \tau_j = t \text{ in sentence } x_n, n = 1, \dots, N\} \quad (8)$$

(as a multiset—duplicates matter and represent frequency).

From this multiset, build:

- The set of unique tuples: $\mathcal{T}_t = \{\mathbf{t}^{(1)}, \mathbf{t}^{(2)}, \dots, \mathbf{t}^{(K_t)}\}$.
- The frequency of each unique tuple: $f^{(i)} = |\{n, j : \mathbf{t}_{n,j} = \mathbf{t}^{(i)}\}|$.

5.4.6 Step 4 — The Embedding Formula

The initial embedding for t is computed as a **two-level weighted average**:

$$\mathbf{e}_t = \sum_{i=1}^{K_t} f^{(i)} \cdot \underbrace{\frac{1}{|\mathbf{t}^{(i)}|} \sum_{t' \in \mathbf{t}^{(i)}} \mathbf{e}_{t'}}_{\text{within-tuple mean}} \quad (9)$$

In words:

- **Inner mean**: average the source embeddings within each unique tuple $\mathbf{t}^{(i)}$ —this gives one candidate decomposition’s vector.
- **Outer sum**: combine these candidate vectors across all observed tuples, weighted by how often each tuple was observed in D .

5.4.7 Worked Example

Suppose the new token $\tau = \text{“_super”}$ (added to V_{new}) appears in 6 sentences of D . Running T_s on each of those sentences and extracting the slice covering “_super”’s span yields:

Sentence	T_s decomposition of “_super”’s span
x_1 (“the supercar arrived”)	(_s, uper)
x_2 (“she’s super smart”)	(_s, uper)
x_3 (“supervise the team”)	(_su, per)
x_4 (“superior choice”)	(_s, uper)
x_5 (“be superstitious”)	(_su, per)
x_6 (“the word super”)	(_ , super)

After Step 3, the data structures for $\tau = \text{“_super”}$ are:

$$\mathcal{T}_{\text{_super}} = \{(_s, \text{uper}), (_su, \text{per}), (_ , \text{super})\} \quad (10)$$

$$\mathcal{F}_{\text{_super}} = (3, 2, 1) \quad (11)$$

Applying the formula:

$$\mathbf{e}_{\text{_super}} = 3 \cdot \frac{1}{2}(\mathbf{e}_{_s} + \mathbf{e}_{\text{uper}}) + 2 \cdot \frac{1}{2}(\mathbf{e}_{_su} + \mathbf{e}_{\text{per}}) + 1 \cdot \frac{1}{2}(\mathbf{e}_{_} + \mathbf{e}_{\text{super}}) \quad (12)$$

The dominant decomposition $(_s, \text{uper})$ contributes 3 units of weight; the less common decompositions contribute proportionally less.

5.4.8 Why Align Outperforms Mean

Mean would have used only **one** decomposition—whatever T_s produces when fed “_super” in isolation, say $(_s, \text{uper})$ —and weighted it uniformly:

$$\mathbf{e}_t^{\text{Mean}} = \frac{1}{2}(\mathbf{e}_{_s} + \mathbf{e}_{\text{uper}}) \quad (13)$$

Align uses **all observed decompositions**, weighted by their empirical prevalence in the actual corpus. This captures three pieces of information Mean discards:

1. **Multiple plausible segmentations exist**: the same span can be segmented differently in different contexts.

2. **Which decomposition is typical:** frequency weighting reflects what the source tokenizer “actually does” with this span in real text.
3. **Context-dependent variation:** BPE/unigram tokenization depends on surrounding characters, so the in-isolation decomposition (what Mean uses) may not match what happens in actual sentences.

5.4.9 Edge Cases

A new token appears only once: $\mathcal{T}_t$ has one tuple, $\mathcal{F}_t = (1)$. Align reduces to a single weighted decomposition.

A new token’s span always decomposes the same way: $\mathcal{T}_t$ has one unique tuple but with high frequency f . The result equals f times the within-tuple mean—same direction as Mean would produce, scaled by frequency. (The magnitude scaling is typically harmless since training quickly normalizes it.)

Span doesn’t align cleanly to source-token boundaries: real text can produce cases where source tokens covering a new token’s span also extend slightly beyond it. The paper’s implementation handles this by recording the smallest source-token sequence that covers the span, adding minor noise but not breaking the algorithm.

5.4.10 Why Align Wins in Low-Resource Settings

FOCUS needs a separately-trained fastText auxiliary model, which itself underfits when the target corpus is small. Align needs only:

- The target corpus D (which you already have for continued pretraining).
- The two tokenizers T_s and T_t (deterministic, already constructed).
- The source embeddings (already in the model).

No auxiliary model is trained, so there is no auxiliary-model-quality bottleneck. The “frequencies” extracted by Align are computed by running two tokenizers on the corpus—a deterministic, scale-invariant operation that works at any data scale.

This is why Yamaguchi found Align outperforms both Random and FOCUS in the low-resource regime: Align’s signal extraction doesn’t depend on training a second model whose own quality requires data.

5.4.11 Algorithm Summary

Algorithm 1 Align Initialization

Require: Source tokenizer T_s , target tokenizer T_t , corpus D , source embeddings $\{\mathbf{e}_{t'}\}_{t' \in V_s}$

Ensure: Initial embedding $\mathbf{e}_t$ for each $t \in V_{\text{new}}$

```

1: Initialize empty record list  $R$ 
2: for each sentence  $x \in D$  do
3:    $\text{target\_seg} \leftarrow T_t(x)$ 
4:    $\text{source\_seg} \leftarrow T_s(x)$ 
5:   for each position  $j$  where  $\text{target\_seg}[j] \in V_{\text{new}}$  do
6:      $\text{span} \leftarrow \text{character\_span}(\text{target\_seg}[j])$ 
7:      $\text{source\_tuple} \leftarrow \text{slice of source\_seg covering same span}$ 
8:     Append  $(\text{target\_seg}[j], \text{source\_tuple})$  to  $R$ 
9:   end for
10: end for
11: for each new token  $t \in V_{\text{new}}$  do
12:   Group records in  $R$  by  $t$  to get multiset  $\mathcal{C}_t$ 
13:   Compute unique tuples  $\mathcal{T}_t$  and frequencies  $\mathcal{F}_t$ 
14:    $\mathbf{e}_t \leftarrow \sum_{i=1}^{K_t} f^{(i)} \cdot \frac{1}{|\mathbf{t}^{(i)}|} \sum_{t' \in \mathbf{t}^{(i)}} \mathbf{e}_{t'}$ 
15: end for
16: return  $\{\mathbf{e}_t\}_{t \in V_{\text{new}}}$ 

```

5.4.12 Practical Recommendation for Coptic/Syriac

For our project, Align is well-suited because:

- We have a moderate but not abundant corpus (tens of MB), so FOCUS’s auxiliary fastText model may underfit.
- Tokenization variation for Coptic/Syriac under Glot500-m’s tokenizer is significant (these scripts likely byte-fallback partially), giving Align rich corpus signal.
- Implementation is straightforward—no auxiliary models, just two tokenizer passes per sentence.

The main implementation effort is the character-span alignment between T_t and T_s outputs, which requires careful tracking of byte/character positions in both segmentations.

5.5 When Initialization Strategy Matters

Yamaguchi et al. (2024) [2] systematically compared these strategies. Key finding:

Yamaguchi’s key finding

With *small* target-language data ($\approx 0.01\text{GB}$), careful initialization (Align, FOCUS, Mean) is crucial because the model cannot recover from random initialization. With *large* data, initialization choice matters less because the model can adapt.

For the Coptic/Syriac project, where data is moderate (tens of MB to a few hundred MB), Mean or Align initialization is recommended over random.

6 Continued Pretraining: Full Fine-Tuning vs. LoRA

6.1 Glot500’s Approach: Full Fine-Tuning

Glott500 trains all parameters of XLM-R-base during continued pretraining:

- **Optimizer:** Adam with $\beta_1 = 0.9$, $\beta_2 = 0.999$.
- **Learning rate:** 5×10^{-5} .
- **Batch size:** 384 samples per step.
- **Hardware:** 8× NVIDIA RTX A6000 GPUs.
- **Training time:** 2 weeks.
- **Objective:** MLM (Masked Language Modeling).

No LoRA was used.

6.2 Why Glot500 Chose Full Fine-Tuning

1. **Abundant data:** Hundreds of GB. The “LoRA Learns Less and Forgets Less” paper (Biderman et al., 2024) confirms that in continued pretraining regimes, full fine-tuning outperforms LoRA even at high ranks.
2. **Moderate model size:** 270M parameters fits comfortably in multi-GPU memory.
3. **Sufficient compute:** 8 GPUs for 2 weeks.
4. **Deep representation change:** Adding 500+ languages requires substantial updates to the model’s internal representations, not just light task adaptation.

6.3 Yamaguchi’s Approach: LoRA + Embedding Full Update

Yamaguchi et al. (2024), targeting single-language adaptation with very limited data (≈ 10 MB), uses:

- LoRA rank 8 on transformer layers.
- Full update on embedding matrix and LM head (via `modules_to_save`).

6.4 Decision Matrix for the Coptic/Syriac Project

Your situation:

- Moderate data (tens of MB to potentially ≈ 100 MB).
- Single RTX A6000.
- Need to preserve performance on Glot500-m’s 511 existing languages.
- Need to add 2 new languages with new vocabulary.

Recommended hybrid approach: LoRA on transformer layers (prevents forgetting, saves compute) + full update on embeddings and LM head (necessary for new tokens to be properly learned).

```

1 from peft import LoraConfig, get_peft_model
2
3 lora_config = LoraConfig(
4     r=8,
5     lora_alpha=32,
6     target_modules=["query", "value", "key", "dense"],
7     lora_dropout=0.05,
8     modules_to_save=["embeddings", "lm_head"], # Full update for these
9 )
10
11 model = get_peft_model(glot500_model, lora_config)

```

Listing 2: Hybrid LoRA configuration

6.5 Training Data Mixture

For continued pretraining, mix Coptic and Syriac with a sample of Glot500-c using $\alpha = 0.3$ multinomial sampling. Including related languages (Arabic for Syriac, Greek for Coptic) can help via shared script and vocabulary.

7 SimAlign: Word-Level Cross-Lingual Alignment

7.1 The Idea

SimAlign uses contextualized embeddings from a pretrained multilingual model to align words across parallel sentences. Unlike classical IBM Models or `fast_align`, it requires no training—no latent variable model, no EM algorithm, no parameters of its own.

7.2 The Algorithm

Algorithm 2 SimAlign: Word Alignment via Contextualized Embeddings

Require: Source sentence s_1 , target sentence s_2 , pretrained multilingual model M , layer index ℓ (default $\ell = 8$)

Ensure: Set of alignment pairs $\mathcal{A} \subseteq \{1, \dots, |s_1|\} \times \{1, \dots, |s_2|\}$

```

1: function SIMALIGN( $s_1, s_2, M, \ell$ )
2:    $\mathbf{E}_1 \leftarrow M(s_1).hidden\_states[\ell]$  ▷ Contextualized embeddings for  $s_1$ 
3:    $\mathbf{E}_2 \leftarrow M(s_2).hidden\_states[\ell]$  ▷ Contextualized embeddings for  $s_2$ 
4:    $\mathbf{S} \leftarrow \text{CosineSimilarity}(\mathbf{E}_1, \mathbf{E}_2)$  ▷  $S_{ij} = \cos(\mathbf{E}_1^{(i)}, \mathbf{E}_2^{(j)})$ 
5:    $align_{12}[i] \leftarrow \arg \max_j S_{ij}$  for each  $i \in \{1, \dots, |s_1|\}$ 
6:    $align_{21}[j] \leftarrow \arg \max_i S_{ij}$  for each  $j \in \{1, \dots, |s_2|\}$ 
7:    $\mathcal{A} \leftarrow \emptyset$ 
8:   for  $i = 1$  to  $|s_1|$  do
9:      $j \leftarrow align_{12}[i]$ 
10:    if  $align_{21}[j] = i$  then ▷ Intersection condition (mutual best match)
11:       $\mathcal{A} \leftarrow \mathcal{A} \cup \{(i, j)\}$ 
12:    end if
13:  end for
14:  return  $\mathcal{A}$ 
15: end function

```

7.3 Why the Intersection Condition

The intersection of argmaxes is a *conservative symmetrization*: a token in s_1 is aligned to a token in s_2 only if both are each other’s best match. This produces fewer but more reliable alignments than either direction alone.

7.4 Why SimAlign Works

The quality of SimAlign’s alignments depends entirely on the quality of the underlying multilingual model’s contextualized representations. If the model has good cross-lingual alignment in its embedding space, SimAlign produces good word alignments. The “latent variable inference” that classical aligners do explicitly is, in SimAlign, *amortized into the multilingual pretraining*.

8 How Does Cross-Lingual Alignment Actually Work?

A common claim in the field is that “shared parameters automatically align languages.” This is an oversimplification. Let us examine the actual mechanisms.

8.1 The Distributional Hypothesis Reformulated

Tokens appearing in similar contexts get similar representations because shared parameters predict them. But for separate corpora in different languages (e.g., English and Korean texts that never appear together), this alone does not explain alignment. Something must bridge the corpora.

8.2 The Bridging Mechanisms

8.2.1 Real anchor tokens

For English-Korean, the actual anchors include:

- Digits 0-9 (Korean uses Arabic numerals: “2024 3” for “March 2024”).
- URLs (<https://example.com>).
- Latin-script tokens in Korean text: Apple, Google, iPhone, MP3, DNA, AI, USB, CEO.
- Code, scientific symbols, romanized loanwords.

Things that are *not* anchors:

- “Steve Jobs” vs. its Korean transliteration in Hangul (different scripts, different tokens, completely independent embeddings).
- “Tokyo” vs. its Korean spelling in Hangul when Korean uses native script.

8.2.2 Subword overlap

When SentencePiece produces literally identical subwords across languages (`_inter`, `_national`, `_de` in Latin-script European languages), embeddings are tied. Etymologically related words tokenized differently (English “ology” vs. French “olog”) have *independent* embeddings—the model has no access to etymology, only token IDs.

For English-Korean, subword overlap is minimal.

8.2.3 Incidental bilingual content

Code-switched text, Wikipedia glosses where multiple scripts appear together (e.g., “Tokyo (with kanji and romanization)”), language-learning material, translated content preserving English terms, EU bilingual documents. Briakou et al. (2023) found that even 1.4% bilingual content in PaLM’s pretraining contained 30M+ translation pairs, and ablating it caused large translation degradation.

8.2.4 Shared parameters

A single $401\text{K} \times 768$ embedding matrix and one transformer for all languages create *pressure* for alignment but do not automatically produce it without bridging signals.

8.2.5 Structural isomorphism

Similar shapes within each language’s distributional structure arise from shared real-world referents. But having similar shapes does not put them in the same coordinate system without anchor tokens to bridge.

8.3 Implications for Coptic and Syriac

- **English-French:** Strong alignment. Extensive shared vocabulary, abundant bilingual content.
- **English-Korean:** Moderate. Limited script overlap, some Latin-script anchors in technical text.
- **English-Coptic:** Poor. Almost no bridging.
- **English-Syriac:** Poor. Different script, limited modern bilingual content.

Empirical alignment scores typically follow this gradient.

8.4 Open Research Questions

The field lacks a rigorous causal account of cross-lingual alignment. Experimental designs to test mechanisms:

1. Anchor token ablation (Dufter & Schütze, 2020, did this for mBERT).
2. Subword overlap ablation: per-language tokenizers.
3. Bilingual content filtering (Briakou et al., 2023).
4. Joint training vs. monolingual + post-hoc alignment (MUSE/VecMap).
5. Mechanistic interpretability (Tang et al., 2024, language-specific neurons).

9 Evaluation: Glot500’s Four Tasks

Glot500-m is evaluated on four tasks using zero-shot or representation-level evaluation.

9.1 Roundtrip Alignment

Word-level cross-lingual alignment via SimAlign on Bible parallel texts. No fine-tuning. Run 5 times with different intermediate languages L_2, L_3, L_4 and report averages.

9.2 Sequence Labeling

Named Entity Recognition (WikiANN) and Part-of-Speech tagging (UD v2.11). Zero-shot: fine-tune on English labeled data, evaluate on target language test sets without further training.

9.3 Sentence Retrieval

Use Tatoeba and Bible parallel sentences. Compute cosine similarity on average word embeddings from layer 8. Report top-10 accuracy.

9.4 Text Classification

Taxi1500 task. Zero-shot from English.

9.5 What “Zero-Shot” Means Here

Zero-shot clarified

“Zero-shot” means zero *labeled* examples in the target language during fine-tuning. The model has still seen the target language during pretraining. This tests cross-lingual representation transfer.

10 Putting It Together: The Coptic/Syriac Project

10.1 Workflow

1. Data collection:

- Coptic Scriptorium + Coptic Bible + patristic texts.
- Peshitta + Syriac patristic literature.
- Target: 30K+ sentences per language.

2. Preprocessing:

Unicode normalization, sentence segmentation, Glot500 cleaning filters (SF1-SF5, CF1-CF2).

3. Auxiliary tokenizer:

SentencePiece unigram, $\sim 10K$ vocab, `character_coverage=1.0`.

4. Vocabulary merging:

Add new tokens to Glot500-m’s tokenizer.

5. Embedding initialization:

Mean or Align (not random, due to limited data).

6. Continued pretraining:

Hybrid LoRA (transformer LoRA + embeddings/LM head full update) on Coptic + Syriac + Glot500-c sample, with $\alpha = 0.3$ multinomial sampling.

7. Evaluation:

- Roundtrip Alignment on Bible parallels.
- Sentence Retrieval.
- Measure forgetting on Glot500’s 511 languages.
- Cross-lingual transfer: Coptic-Greek/English, Syriac-Arabic/Hebrew.

10.2 Experimental Design

Compare four setups:

- (A) Full fine-tuning, Coptic+Syriac only.
- (B) Full fine-tuning, Coptic+Syriac + Glot500-c sample.
- (C) Hybrid LoRA, Coptic+Syriac only.
- (D) Hybrid LoRA, Coptic+Syriac + Glot500-c sample.

This reveals the interaction between training method (full vs. LoRA), data composition (target-only vs. mixed), and outcomes (target performance vs. forgetting).

10.3 Project Framing

Project contribution

This work makes three contributions:

1. **Methodological critique:** Glot500’s automatic collection pipeline missed Coptic and Syriac despite sufficient available data. We document this gap and propose academic-source-based collection as a complement.
2. **Practical extension:** We extend Glot500-m to two new low-resource languages with substantial historical and religious importance.
3. **Empirical comparison:** We compare full fine-tuning vs. hybrid LoRA in the moderate-data regime characteristic of academic language collection efforts.

11 Summary and Key Takeaways

1. **Glott500-m covers 511 languages but not Coptic or Syriac**, primarily due to automatic pipeline limitations rather than data scarcity. Both languages have enough academic text to exceed Glot500’s 30K sentence threshold.
2. **Multinomial sampling with $\alpha = 0.3$** balances head and tail languages. The notation $p(L_i)$ in papers is sloppy; $p(L = L_i)$ or π_i is more precise. Individual sentences from large corpora are sampled *less* often than from small corpora (exponent $\alpha - 1 = -0.7$), even though large-corpus languages are sampled more often overall.
3. **Vocabulary extension follows a clear pipeline:** train auxiliary tokenizer on new data, merge into existing vocabulary, resize embeddings, initialize.
4. **Embedding initialization matters when data is limited.** With small data, use Mean, Align, or FOCUS rather than random.
5. **Glott500 used full fine-tuning, not LoRA**, because of abundant data, moderate model size, and ample compute. For smaller-scale extensions like Coptic/Syriac, hybrid LoRA (transformer LoRA + embedding full update) is appropriate.
6. **SimAlign requires no training** and produces alignments via intersection of argmaxes on contextualized embeddings. Quality depends entirely on the underlying model’s cross-lingual representations.

7. **Cross-lingual alignment mechanisms are limited and indirect:** anchor tokens, subword overlap, incidental bilingual content, shared parameters, structural isomorphism. The field lacks a rigorous causal account, and the “shared parameters automatically align languages” story is overstated.
8. **Evaluation should measure both new-language performance and forgetting** on existing languages. The project’s framing as a methodological complement to automatic collection strengthens its academic contribution.

References

References

- [1] ImaniGooghari, A., Lin, P., Kargaran, A. H., et al. (2023). *Glott500: Scaling Multilingual Corpora and Language Models to 500 Languages*. ACL 2023. <https://arxiv.org/abs/2305.12182>
- [2] Yamaguchi, A., Villavicencio, A., & Aletras, N. (2024). *How Can We Effectively Expand the Vocabulary of LLMs with 0.01GB of Target Language Text?* Computational Linguistics (to appear). <https://arxiv.org/abs/2406.11477>
- [3] Conneau, A., Khandelwal, K., Goyal, N., et al. (2020). *Unsupervised Cross-lingual Representation Learning at Scale*. ACL 2020.
- [4] Dobler, K., & de Melo, G. (2023). *FOCUS: Effective Embedding Initialization for Monolingual Specialization of Multilingual Models*. EMNLP 2023.
- [5] Hu, E. J., Shen, Y., Wallis, P., et al. (2021). *LoRA: Low-Rank Adaptation of Large Language Models*. <https://arxiv.org/abs/2106.09685>
- [6] Biderman, D., Portes, J., Ortiz, J. J. G., et al. (2024). *LoRA Learns Less and Forgets Less*. <https://arxiv.org/abs/2405.09673>
- [7] Jalili Sabet, M., Dufter, P., Yvon, F., & Schütze, H. (2020). *SimAlign: High Quality Word Alignments without Parallel Training Data using Static and Contextualized Embeddings*. EMNLP Findings 2020.
- [8] Briakou, E., Cherry, C., & Foster, G. (2023). *Searching for Needles in a Haystack: On the Role of Incidental Bilingualism in PaLM’s Translation Capability*. ACL 2023.
- [9] Dufter, P., & Schütze, H. (2020). *Identifying Elements Essential for BERT’s Multilinguality*. EMNLP 2020.
- [10] Tang, T., Luo, W., Huang, H., et al. (2024). *Language-Specific Neurons: The Key to Multilingual Capabilities in Large Language Models*. <https://arxiv.org/abs/2402.16438>